CSCI 49900: Midterm Report

Alex Haskel
alex.haskel98@login.cuny.edu

James Lacoste
james.lacoste56@login.cuny.edu

Kailun You
kailun.you08@login.cuny.edu

I. Project Overview

The Language Model Lab project is an interactive platform meant to demonstrate the mechanics of Large Language Model (LLM) inference optimization and model behavior through internal interventions without the need for special tools or programming.

II. Current Progress and Implementation Status

Currently, we have a viable prototype for the platform consisting of a Javascript, HTML, and CSS frontend connected to a Python backend.

The backend API is written in FastAPI and Pydantic. It correctly accepts prompts, decoding parameters, and intervention parameters from the user, allowing for the creation of two side-by-side configurations for them to compare. The backend also implements concurrency handling and custom error behavior.

The backend currently supports:

- Decoding parameters: max new tokens, sampling (determines whether the model uses probabilistic sampling or deterministic decoding), temperature, top-K, top-P, and repetition penalty.
- Thinking mode
- Optional system prompt
- Intervention parameters: MLP unit and attention head ablation.

The frontend applies a first layer of constraints to these parameters. The user's input is then authoritatively re-validated in the backend by Pydantic, while the intervention settings are validated within the backend functions.

The backend currently loads a standard Qwen3 0.6B model, queries it with the user's input, and returns generated text and metrics like latency and tokens generated per second. The front end is able to display the text and metrics returned by the backend side by side.

The other side of the project involves letting the user choose among different precision formats and optimized kernels, compare, and learn about them. The kernel development is going smoothly. We have identified which weights to target for optimization by determining whether there would be a large benefit in compressing them. The main criteria used for this was the range between the largest and smallest weight magnitudes in a particular matrix row. We then quantized the model by replacing standard FP16 (16-bit floating point) values with scaled INT8 (8-bit integer) values.

The optimized kernel is under development. We are implementing an improved general matrix vector multiplication function. This operation is called by the Python backend but performed by the C++ CUDA kernel. Since our optimized models are using quantized weights, the GPU will need to spend less time reading memory and will be able to store more information in its caches.

III. Changes from Original Proposal

So far, we have stuck to the original proposal quite closely. Model quantization and kernel development are well-studied areas in machine learning, so we had a good idea of what to expect going into the project on this front.

The intervention platform was purposefully planned as an incremental project with optional checkpoints, as time permits. The basic MLP and attention interventions have been correctly implemented. More advanced interventions, such as activation steering or direct intervention of model weights, remain a future goal.

On the frontend side, one technical change was using plain HTML, CSS and Javascript instead of React. We also de-emphasized the use of visuals compared to our initial proposal, choosing to focus primarily on the user's ability to interact with the models in a large variety of ways, and keeping the platform as responsive as possible. We had originally planned to use React and D3.js to create visuals, but found it simpler to make this switch for the time being.

IV. Evaluation and Testing

Our backend is regularly tested through a custom test suite using pytest. We wrote a variety of test files that check for correct text generation, model intervention handling, concurrency locks, schema validation logic, and API responses. When necessary, we used mock objects and patched functions to avoid loading large models, while replicating the behaviors triggered by an end user without performing expensive operations. The backend API is also regularly inspected more informally through direct end-to-end generation. The frontend was locally tested using Live Server and the browser console. We tried a variety of inputs to verify that each was read correctly and included in the generated API request.

For our kernel work, we used NVIDIA Nsight Systems and NVIDIA Nsight Compute for profiling and evaluation. NVIDIA Nsight Systems was mostly used for high-level evaluation as it records GPU activity over entire operations and sequences of operations. We used NVIDIA Nsight Compute to test specific aspects and operations of the kernel. We also used AI-assisted code reviews to inspect implementation quality.

V. Individual Contributions

Alex worked on the backend that allows the end user to interact with our chosen model, which is of the Qwen3 family. He wrote the API that allows the frontend to interact with the model, including its custom error behavior and concurrency handling. He also wrote the functions that load the model, the text generation utility functions, and the functions that measure various model inference metrics. In addition, he designed the schemas that determine the API contract between the frontend and backend, as well as the intervention logic that allows users to modify aspects of the model in order to compare responses. Alex is also responsible for coming up with ideas for new parameters and controls for users to experiment with.

James worked on the kernel development and model optimization aspects of our project. This involved developing a custom accelerator program to perform highly optimized instructions in parallel. This will allow us to maximize performance for our chosen model and learn more about how AI systems operate on hardware. James developed this kernel in C++ using the CUDA framework for NVIDIA GPUs. To call the functions and run wrappers, he used PyTorch. This will allow us to interact with the multidimensional arrays that store the model's weights. James also performed the integration of our initially separate backend and quantization directories into a single repository.

The choice of using Qwen3 models was determined by Alex and James after a series of discussions about ease of use, model size, and model responsiveness. Alex performed a series of comparative experiments in Jupyter notebooks to determine which model family produced more interesting responses, while James performed initial performance profiling on these models.

Kailun built the frontend. He used HTML for the layout, CSS for the styling and JavaScript for behavior. He wrote the JavaScript code that collects the user's inputs, builds a JSON request, and sends it to the backend. The change from React to plain HTML was done to avoid conflicts with D3.js visualizations.

VI. Future Work and Updated Plan

We plan to further develop our project to make model interaction more interesting to the end user. One projected change is streaming output. Currently, the backend accepts a user request, calls the model using these parameters, and returns generated text in a single HTTP response. We plan to stream output so that the backend gradually returns small parts of the generated text as they become available. This will allow the frontend to display the output gradually in real time as it becomes available, instead of displaying a loading state until the entire response has been generated.

Another change we are currently evaluating is allowing for turn-based conversations. This will require the frontend to send the entire conversation history from the current session back to the backend with each new request, instead of treating each single prompt independently. This may make interaction more interesting, and may require adding further controls, such as context window size.

So far, our inference metrics include number of tokens generated, latency, and tokens per second. We may add more informative metrics, such as time to first token, or decode time per token. Additional interventions, like those mentioned in Section III, remain a stretch goal if time permits.

For our kernel backend, we are considering additional optimization techniques such as kernel fusion or memory pre-allocation. We will collaborate and brainstorm ways in which we can intuitively display this optimization work on our website. We are also currently working on incorporating model quantization choice as yet another experimental parameter.

Before the end of the semester, we also intend to add more visual displays to the frontend, possibly using D3.js, as well as more sophisticated displays for failed or error states.

Depending on deployment requirements for the final demo, we may also investigate GPU hosting and deployment options for making the platform publicly accessible, although this remains an optional goal.